

JavaScript Assignment 3

Objects, Prototypes & Classes — Tricky Edition (User / Auth Theme)

Follow-up to today's session (19th July 2026, Terralogic Academy). Complete the following 5 tasks, built around a small user-authentication system: creating users, logging in/out, and managing sessions. Each task maps to its own file, as noted below. Several steps are intentionally tricky — edge cases, shallow-copy traps, and behavior that looks like a bug but isn't. Use `console.log` to show the required output for every step, and add short comments explaining the tricky parts.

#	Assignment	File	Topic
1	Object Creation Patterns	Users.js	Literal vs Factory vs Constructor vs Class + shared-method pitfalls
2	<code>Object.assign</code> & <code>Object.freeze</code>	UserUtils.js	Merging user settings, shallow-merge/freeze traps
3	Getters & Setters	authGetterSetter.js	Validated login state, computed getters, getter-only errors
4	Prototypal Inheritance	authPrototypes.js	<code>Object.create</code> , shared login/logout methods, shadowing
5	Classes, <code>super()</code> , Private Fields	authInheritance.js	Session polymorphism, private password field, static counters

Assignment 1 — Object Creation Patterns (Tricky)

Maps to: Users.js | Topic: Object Literal vs Factory vs Constructor vs ES6 Class, applied to user records

- 1 Create a single user object using an object literal: `{ username: "arjun", password: "pass123" }`.
- 2 Create a factory function `createUser(username, password)` that returns a user object with a method `login(input)` that returns `input === password`. Use it to build an array of users from: `[["neha", "abc123"], ["ravi", "qwerty"], ["sara", "hunter2"]]`.
- 3 Create a constructor function `User(username, password)` using `this`, but define `login` on `User.prototype` (not inside the constructor). Build the same array using `new`. Tricky: log `Object.getPrototypeOf(user1) === User.prototype` to prove the method is shared, not copied per instance.
- 4 Create an ES6 class `User` with a constructor for `username` and `password`, plus a static method `User.isValidUsername(name)` that returns `true` only if `name.length >= 4`. Build the array again, then call `User.isValidUsername()` on a couple of usernames.
- 5 Tricky twist: try calling the class as a plain function — `User("test", "1234")` without `new` — wrap it in `try/catch` and log the `TypeError` message you get.
- 6 Log all 4 approaches. For each, write one line explaining when you would use it, plus one extra line explaining why factory-created users each get their own separate copy of `login` in memory, while constructor-function and class-created users all share one copy via the prototype.

Assignment 2 — `Object.assign` & `Object.freeze` (Tricky)

Maps to: UserUtils.js | Topic: Merging account settings and locking a user profile

- 1 Create `defaultSettings = { theme: "light", notifications: { email: true, sms: false } }`.
- 2 Create `userPreferences = { theme: "dark", notifications: { sms: true } }`.

- 3 Merge them with `Object.assign({}, defaultSettings, userPreferences)` into `finalSettings`.
- 4 Log `finalSettings`. Tricky: notice that `notifications` was completely replaced, not merged — `email: true` is gone. Add a comment explaining that `Object.assign` only does a **shallow** copy, so nested objects overwrite each other entirely.
- 5 Fix it by also spreading the nested objects: build `fixedSettings` using `Object.assign({}, defaultSettings, userPreferences, { notifications: {...defaultSettings.notifications, ...userPreferences.notifications} })`. Log it and confirm both `email: true` and `sms: true` are present.
- 6 Call `Object.freeze(fixedSettings)` to represent a "locked" user profile that shouldn't change after account creation.
- 7 Add `"use strict"` at the top of the file. Try `fixedSettings.theme = "blue"` inside a `try/catch` — it should throw a `TypeError` in strict mode. Log the caught error message.
- 8 Tricky: now try `fixedSettings.notifications.sms = false`. This will succeed and change the value, even though `fixedSettings` is frozen. Log `fixedSettings.notifications` afterward to prove the nested object was mutated, and add a comment explaining that `Object.freeze` is shallow — it only locks the top-level properties, not nested objects.

Assignment 3 — Getters & Setters (Tricky)

Maps to: `authGetterSetter.js` | Topic: Validated login attempts and a computed, getter-only session status

- 1 Create a class `Account` with constructor (`username`) storing `username`, an internal `_failedAttempts = 0`, and `_isLoggedIn = false`.
- 2 Add a method `login(password, correctPassword)`: if `password !== correctPassword`, increment `_failedAttempts` and log `"Incorrect password"`; if `_failedAttempts` reaches 3, log `"Account locked"` instead of checking the password at all, even if it is correct now. Otherwise, on a correct password, set `_isLoggedIn = true` and reset `_failedAttempts = 0`.
- 3 Add a setter `set failedAttempts(value)` — if `value < 0`, log `"Failed attempts cannot be negative"` and return; otherwise store it (used internally, but should still be safe if called directly).
- 4 Add a getter `get status()` that returns `"Locked"` if `_failedAttempts >= 3`, otherwise `"Logged in"` or `"Logged out"` based on `_isLoggedIn` — a getter computed purely from other internal fields, with no field of its own.
- 5 Create `const acc = new Account("neha")`. Try logging in with the wrong password twice, logging `acc.status` after each attempt.
- 6 Try a third wrong attempt → account should lock. Log `acc.status` → should be `"Locked"`. Tricky: now try logging in with the **correct** password anyway → it should still fail because the account is locked, proving the lockout overrides even a correct password.
- 7 Tricky: since `status` has no setter, try (inside a `try/catch`, in `"use strict"` mode) assigning `acc.status = "Logged in"` directly. It should throw. Log the caught error message, then log `acc.status` again to confirm it did not change.

Assignment 4 — Prototypal Inheritance with `Object.create` (Tricky)

Maps to: `authPrototypes.js` | Topic: `Object.create`, shared login/logout behavior, and property shadowing

- 1 Create a plain object `sessionHandler` with a shared property `role`: `"guest"` and two methods: `login`: `function() { return `${this.username} logged in as ${this.role}`; }` and `logout`: `function() { return `${this.username} logged out`; }`.
- 2 Create `user1 = Object.create(sessionHandler)`, set `user1.username = "arjun"` (no own `role` property).
- 3 Create `user2 = Object.create(sessionHandler)`, set `user2.username = "sara"`, and also set `user2.role = "admin"` — this creates an **own** property that shadows the prototype's `role`.
- 4 Call `user1.login()` and `user2.login()` and log both. Explain in a comment why `user1` logs in as `"guest"` (read from the prototype) while `user2` logs in as `"admin"` (read from its own property, which takes priority).
- 5 Tricky: after the above, set `sessionHandler.role = "member"` and call `user1.login()` again. Log the new result and explain why it changes to `"member"` (`user1` has no own `role`, so it always looks up the prototype live) while `user2.login()` still says `"admin"` and is unaffected.
- 6 Log `Object.keys(user1)` next to `Object.getOwnPropertyNames(Object.getPrototypeOf(user1))` to show that `login`, `logout`, and `role` actually live on the prototype object, not on `user1` itself.
- 7 Add a comment explaining why `login/logout` are not duplicated in memory for every user (prototype chain), and how this differs from the factory-function pattern in Assignment 1, where each user got its own separate copy of the method.

Assignment 5 — ES6 Classes, `super()`, Private Fields, Polymorphism (Tricky)

Maps to: `authInheritance.js` | Topic: Session inheritance, encapsulated passwords, static counters, and chaining `super` calls

- 1 Create a parent class `User` with constructor (`username`) storing `this.username`, a static field `static totalUsers = 0` that increments by 1 every time any `User` (including subclasses) is constructed, and a method `logout()` that logs ``${this.username} logged out``.
- 2 Create a child class `AdminUser` extends `User`: add a private field `#password`; constructor takes (`username`, `password`), calls `super(username)`, sets `this.#password`; add `login(attempt)` which logs `"Access granted"` if `attempt === this.#password`, otherwise logs `"Access denied"`; add a getter `hasPassword` that returns `true/false` without ever exposing `#password` itself; override `logout()` so it first calls `super.logout()` and then logs `"(admin session cleared)"`.
- 3 Create another child class `GuestUser` extends `User`: constructor calls `super(username)`; override `logout()` to first call `super.logout()` and then log `"(guest data discarded)"` — tricky: you must call `super.logout()` rather than re-writing the parent's log line yourself.
- 4 Create objects: `const admin = new AdminUser("neha", "s3cret")` and `const guest = new GuestUser("visitor1")`.
- 5 Call `admin.login("wrong")` → should log `"Access denied"`. Then call `admin.login("s3cret")` → should log `"Access granted"` (tricky: the wrong attempt first should not lock or affect the later correct one, since this class has no logout logic — contrast with Assignment 3's `Account`).
- 6 Put both users in an array `const sessions = [admin, guest]` and loop through calling `.logout()` on each — demonstrate polymorphism (same method call, different behavior per class).
- 7 Log `User.totalUsers` after both objects are created — it should be 2, proving the static field is shared across the whole class hierarchy rather than duplicated per instance.
- 8 Log `admin.hasPassword` using the getter.

- 9 Add a comment showing that accessing `admin.#password` directly from outside the class would throw a `SyntaxError`.
- 10 Write the expected console output at the bottom as a comment block, in the same style as the previous `inheritance.js`.

Note for students: several steps above are designed to look wrong at first — a merge that silently drops data, a freeze that doesn't fully protect an object, a login lockout that overrides even a correct password, a prototype value that changes after the fact. Don't just log the output — add a one-line comment explaining *why* it behaves that way. That explanation is worth as much as getting the code to run.